UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA

Practica 1

Materia:	Aprendizaje Automático (Machine Learning)
Docente:	M.Sc. Menfy Morales Rios
Estudiante:	Carlos Eduardo Reyes Barja
Fecha:	18 de Febrero de 2026

Resumen de IA’s en el mercado vigente, concepto, objetivo, aplicaciones frecuentes

1. ChatGPT

1.1. Concepto

Es un sistema de chat basado en el modelo de lengua por Inteligencia Artificial GPT, desarrollado por la empresa OpenAI. Emplea procesamiento de lenguaje natural para mantener conversaciones realistas con los usuarios y generar contenido que incluye artículos, resúmenes de textos, consejos y más.
ChatGPT, una de las implementaciones más conocidas de IA conversacional, se ejecuta en los modelos de transformadores generativos preentrenados (GPT) de OpenAI.

1.2. Objetivo

El objetivo de ChatGPT es generar texto similar al humano y entablar conversaciones con usuarios sobre diversos temas, brindando información, asistencia y facilitando interacciones en lenguaje natural. Su objetivo es ser un agente conversacional versátil y útil

1.3. Aplicaciones frecuentes

- Redactar textos complejos, ya sea académicos, periodísticos, informativos
- Traducir textos de un idioma a otro.
- Crear código de programación
- Brindar recomendaciones de cualquier tema (series, música, libros, planes de ocio…)

2. Meta AI

2.1. Concepto

Meta AI es el asistente de inteligencia artificial desarrollado por Meta (la empresa matriz de Facebook, Instagram y WhatsApp). Está basado en su modelo de lenguaje propio, Llama 4 .
Su característica más distintiva es que no es solo una app independiente, sino que está integrada directamente en las aplicaciones que se usa a diario: WhatsApp, Instagram, Facebook y Messenger.

A diferencia de asistentes más "profesionales" como Claude, Meta AI está diseñada para ser una compañera conversacional cercana, creativa y útil en el día a día, pensada para el usuario promedio que ya vive dentro del ecosistema de Meta .

2.2. Objetivo

El objetivo de Meta es que el usuario no salga de su ecosistema para resolver tareas rápidas. Esto incluye búsquedas simples, creación de imágenes generadas con IA y asistencia para revisar textos, en un contexto en el que los chatbots se consolidaron como el canal de consulta inmediata dentro de las apps móviles.

2.3. Aplicaciones frecuentes

- Generación de imágenes dentro del chat
- Traducción rápida de mensajes recibidos
- Revisión y corrección de escritura

3. Gemini

3.1. Concepto

Gemini es el asistente de inteligencia artificial de Google, basado en su familia de modelos Gemini 3, que representa la evolución y unificación de todos los esfuerzos previos de la compañía en IA (como Bard, Duet AI y el clásico Asistente de Google)
A diferencia de otros asistentes que son aplicaciones independientes, Gemini está profundamente integrado en todo el ecosistema de Google: el Buscador, Android, Gmail, Google Docs, Sheets, YouTube y más . No es solo una app que abres; es el motor cognitivo que impulsa la experiencia digital de miles de millones de usuarios.

3.2. Objetivo

Su objetivo principal es mejorar la interacción y comprensión del lenguaje natural mediante capacidades multimodales, es decir, la habilidad para procesar y entender texto, imágenes, audio y otros tipos de datos simultáneamente.
Esto lo hace altamente versátil y potente en diversas aplicaciones, desde la traducción de idiomas hasta la generación de contenido y la programación avanzada.

3.3. Aplicaciones frecuentes

Google no quiere que tengas que abrir una app aparte para usar IA. Su objetivo es que Gemini esté donde ya estás: en tus búsquedas de Google, en tu bandeja de entrada de Gmail, en tus documentos, en tu móvil Android.

Seleccione al menos 3

GENERADOR DE TEXTO: "Generador de Nombres de Productos"

```
# %% [markdown]
# # GENERADOR DE TEXTO: "Generador de Nombres de Productos"
# ## Concepto
# El generador de texto crea nombres creativos para productos combinando
aleatoriamente adjetivos y sustantivos de listas predefinidas. Es útil para
generar ideas de nombres de productos, marcas o campañas de marketing de
manera rápida y automatizada.
#
# **Formato de salida:**
# - Texto en consola
# - Archivo .txt (formato de texto plano UTF-8)
#
# **Características:**
# - Generación aleatoria de combinaciones
# - Interfaz interactiva por consola
# - Exportación a archivo de texto
# - Validación de entrada del usuario

# %%
import random
from datetime import datetime

# %% [markdown]
# ## Librerías utilizadas
#
# ### 1. **random** (Librería estándar)
# - **Propósito**: Generar números aleatorios y hacer selecciones al azar
# - **Funciones principales**:
#   - `random.choice(lista)`: Selecciona un elemento aleatorio de una lista
#   - `random.randint(a, b)`: Genera un número entero aleatorio entre a y b
# - **Ventaja**: No requiere instalación adicional
#
# ### 2. **os** (Librería estándar)
# - **Propósito**: Interactuar con el sistema operativo
# - **Uso en este programa**: Verificar si el archivo de salida ya existe
```

```python
#
# ### 3. **datetime** (Librería estándar)
# - **Propósito**: Manejo de fechas y horas
# - **Uso en este programa**: Agregar timestamp a los archivos generados

# %%
def generar_nombre_producto():
    """
    Genera un nombre aleatorio combinando adjetivo + sustantivo

    Returns:
        str: Nombre generado (ej. "Super Widget")
    """

    adjetivos = [
        "Super", "Mega", "Ultra", "Smart", "Eco", "Tech",
        "Pro", "Premium", "Basic", "Deluxe", "Mini", "Maxi",
        "Hyper", "Power", "Quick", "Easy", "Fast", "Cool"
    ]

    sustantivos = [
        "Widget", "Gadget", "Tool", "Device", "App", "System",
        "Solution", "Box", "Kit", "Pack", "Set", "Bundle",
        "Hub", "Pod", "Core", "Sync", "Link", "Connect"
    ]

    adjetivo = random.choice(adjetivos)
    sustantivo = random.choice(sustantivos)

    return f"{adjetivo} {sustantivo}"

def generar_multiples_productos(cantidad):
    """
    Genera múltiples nombres de productos

    Args:
        cantidad (int): Número de nombres a generar

    Returns:
        list: Lista con los nombres generados
    """

    return [generar_nombre_producto() for _ in range(cantidad)]
```

```python
def guardar_en_archivo(productos, nombre_base="productos_generados"):
    """
    Guarda los productos en un archivo con timestamp

    Args:
        productos (list): Lista de productos a guardar
        nombre_base (str): Nombre base del archivo

    Returns:
        str: Nombre del archivo creado
    """

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    nombre_archivo = f"{nombre_base}_{timestamp}.txt"

    with open(nombre_archivo, 'w', encoding='utf-8') as f:
        f.write("=" * 50 + "\n")
        f.write("PRODUCTOS GENERADOS\n")
        f.write(f"Fecha: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
        f.write("=" * 50 + "\n\n")

        for i, producto in enumerate(productos, 1):
            f.write(f"{i:3d}. {producto}\n")

        f.write(f"\nTotal: {len(productos)} productos")

    return nombre_archivo

def mostrar_productos_en_tabla(productos):
    """
    Muestra los productos en formato tabla para mejor visualización

    Args:
        productos (list): Lista de productos a mostrar
    """

    print("\n" + "=" * 60)
    print("PRODUCTOS GENERADOS".center(60))
    print("=" * 60)

    columnas = 3
    for i in range(0, len(productos), columnas):
```

```python
        fila = productos[i:i+columnas]
        elementos_fila = []
        for j, prod in enumerate(fila):
            num = i + j + 1
            elementos_fila.append(f"{num:2d}. {prod:15s}")
        print("   ".join(elementos_fila))

    print("=" * 60)
    print(f"Total: {len(productos)} productos")

# %%
def main():
    """Función principal del programa"""

    print("=" * 60)
    print("GENERADOR DE NOMBRES DE PRODUCTOS".center(60))
    print("=" * 60)

    while True:
        print("\nOPCIONES:")
        print("   1. Generar nombres")
        print("   2. Ver estadísticas")
        print("   3. Salir")

        opcion = input("\nSelecciona una opción (1-3): ").strip()

        if opcion == "1":
            try:
                cantidad = int(input("   ¿Cuántos nombres generar? "))

                if cantidad <= 0:
                    print("Debe ser un número positivo")
                    continue

                print(f"\nGenerando {cantidad} nombres...")

                productos = generar_multiples_productos(cantidad)

                mostrar_productos_en_tabla(productos)

                guardar = input("\nGuardar en archivo? (s/n): ").lower()
                if guardar == 's':
                    archivo = guardar_en_archivo(productos)
```

```
                print(f"Guardado en: {archivo}")

            except ValueError:
                print("Error: Ingresa un número válido")
            except Exception as e:
                print(f"Error inesperado: {e}")

        elif opcion == "2":
            print("\nESTADÍSTICAS:")
            print(f"   • Combinaciones posibles: 18 x 18 = 324")
            print(f"   • Formato de salida: TXT (UTF-8)")
            print(f"   • Librerías utilizadas: random, os, datetime")

        elif opcion == "3":
            print("\nHasta luego!")
            break

        else:
            print("Opción no válida")

if __name__ == "__main__":
    main()

# %% [markdown]
# ## Conclusiones del Generador de Texto
#
# ### Logros:
# - Implementación exitosa de generación aleatoria de texto
# - Interfaz interactiva amigable
# - Exportación a archivo con timestamp
# - Múltiples formas de visualización
# - Manejo robusto de errores
#
# ### Características implementadas:
# 1. **Aleatoriedad controlada**: Combinaciones siempre coherentes
# 2. **Escalabilidad**: Fácil añadir más palabras a las listas
# 3. **Persistencia**: Guardado automático con fecha/hora
# 4. **UX mejorada**: Tablas, colores ASCII, feedback visual
#
# ### Posibles mejoras futuras:
# - Conectar con API de sinónimos para más variedad
# - Añadir interfaz gráfica simple
# - Implementar generación basada en categorías específicas
```

```
# - Exportar a otros formatos (JSON, CSV, Excel)
```

Corrida

```
============================================================
          GENERADOR DE NOMBRES DE PRODUCTOS
============================================================

OPCIONES:
  1. Generar nombres
  2. Ver estadísticas
  3. Salir

Generando 20 nombres...

============================================================
                PRODUCTOS GENERADOS
============================================================
 1. Pro Device        2. Ultra Kit         3. Hyper Bundle
 4. Cool Tool         5. Tech Set          6. Hyper Box
 7. Cool Connect      8. Mega Tool         9. Cool System
10. Maxi Hub         11. Pro System       12. Power Pack
13. Premium Connect  14. Pro Solution     15. Cool Pod
16. Basic Device     17. Hyper Sync       18. Cool Gadget
19. Maxi Set         20. Mini Box
============================================================
Total: 20 productos

OPCIONES:
  1. Generar nombres
  2. Ver estadísticas
  3. Salir

Hasta luego!
```

GENERADOR DE IMÁGENES: Figuras Geométricas Aleatorias

```
# %% [markdown]
# # GENERADOR DE IMÁGENES: Figuras Geométricas Aleatorias
#
```

```
# ## Concepto
# Este notebook implementa un generador de imágenes que crea composiciones
artísticas
# con figuras geométricas aleatorias. Cada imagen es única y generada
proceduralmente.
#
# **Características:**
# - Generación de imágenes RGB (color verdadero)
# - Figuras: rectángulos, elipses, líneas y polígonos
# - Colores aleatorios en formato RGB
# - Posiciones y tamaños variables
# - Múltiples formatos de salida (PNG, JPG)
#
# **Formato de imagen:** PNG (Portable Network Graphics)
# - **Ventajas:** Sin pérdida de calidad, transparencia, compresión sin
artefactos
# - **Resolución:** 800x600 píxeles (personalizable)
# - **Profundidad de color:** 24 bits (16.7 millones de colores)
#
# **Aplicaciones:**
# - Arte generativo
# - Fondos de pantalla únicos
# - Material para pruebas de visión por computadora
# - Inspiración para diseño gráfico

# %%
import random
import os
import matplotlib.pyplot as plt
from datetime import datetime
import numpy as np
from PIL import Image, ImageDraw, ImageFont


# %% [markdown]
# ## Librerías utilizadas
#
# ### 1. **PIL/Pillow** (Python Imaging Library)
# - **Propósito**: Creación y manipulación de imágenes
# - **Componentes principales**:
#   - `Image`: Clase principal para trabajar con imágenes
#   - `ImageDraw`: Dibuja figuras geométricas sobre imágenes
#   - `ImageFont`: Manejo de fuentes para texto
```

```
# - **Instalación**: `pip install Pillow`
#
# ### 2. **matplotlib.pyplot**
# - **Propósito**: Visualizar imágenes directamente en el notebook
# - **Uso**: Mostrar las imágenes generadas sin guardarlas
# - **Ventaja**: Permite ver resultados inmediatos
# - **Instalación**: `pip install matplotlib`
#
#
# ### 3. **numpy**
# - **Propósito**: Manipulación numérica de matrices de píxeles
# - **Uso**: Para efectos avanzados como degradados
# - **Ventaja**: Operaciones matemáticas eficientes
# - **Instalación**: `pip install numpy`
#
#
# ### 4. **random** (estándar)
# - **Propósito**: Generar valores aleatorios para colores, posiciones,
etc.
# - **Uso**: Cada imagen es única
#
# ### 5. **os** (estándar)
# - **Propósito**: Crear carpetas y manejar archivos
# - **Uso**: Organizar las imágenes generadas
#
# ### 6. **datetime** (estándar)
# - **Propósito**: Timestamps para nombres de archivo únicos

# %%
class GeneradorImagenes:
    """
    Clase para generar imágenes artísticas con figuras geométricas
    """
    def __init__(self, ancho=800, alto=600):
        """
        Inicializa el generador con dimensiones personalizables

        Args:
            ancho (int): Ancho de la imagen en píxeles
            alto (int): Alto de la imagen en píxeles
        """
        self.ancho = ancho
        self.alto = alto
```

```python
        self.colores_fondo = self._generar_paleta_fondos()

    def _generar_paleta_fondos(self):
        """
        Genera una paleta de colores de fondo (método interno)
        """
        return [
            (255, 255, 255),  # Blanco
            (240, 248, 255),  # Alice Blue
            (245, 245, 220),  # Beige
            (230, 230, 250),  # Lavender
            (255, 228, 225),  # Misty Rose
            (240, 255, 240),  # Honeydew
            (255, 250, 205),  # Lemon Chiffon
        ]

    def generar_color_aleatorio(self, brillo_min=0, brillo_max=255):
        """
        Genera un color RGB aleatorio

        Args:
            brillo_min (int): Valor mínimo para cada componente (0-255)
            brillo_max (int): Valor máximo para cada componente (0-255)

        Returns:
            tuple: (R, G, B) valores de color
        """
        return (
            random.randint(brillo_min, brillo_max),
            random.randint(brillo_min, brillo_max),
            random.randint(brillo_min, brillo_max)
        )

    def crear_imagen_vacia(self, color_fondo=None):
        """
        Crea una imagen nueva con el fondo especificado

        Args:
            color_fondo (tuple): Color RGB para el fondo

        Returns:
            PIL.Image: Nueva imagen
        """
```

```python
        if color_fondo is None:
            color_fondo = random.choice(self.colores_fondo)

        # Crear imagen RGB
        imagen = Image.new('RGB', (self.ancho, self.alto), color_fondo)
        return imagen

    def dibujar_rectangulo(self, dibujo):
        """
        Dibuja un rectángulo aleatorio

        Args:
            dibujo (ImageDraw): Objeto para dibujar
        """
        # Coordenadas aleatorias
        x1 = random.randint(50, self.ancho - 150)
        y1 = random.randint(50, self.alto - 150)
        x2 = x1 + random.randint(50, 200)
        y2 = y1 + random.randint(50, 200)

        # Color aleatorio
        color = self.generar_color_aleatorio()

        # Borde aleatorio (0 = sin borde, 1-5 = con borde)
        if random.random() > 0.3:  # 70% con borde
            dibujo.rectangle([x1, y1, x2, y2],
                             fill=color,
                             outline=(0, 0, 0),
                             width=random.randint(1, 5))
        else:
            dibujo.rectangle([x1, y1, x2, y2], fill=color)

    def dibujar_elipse(self, dibujo):
        """
        Dibuja una elipse aleatoria
        """
        x1 = random.randint(50, self.ancho - 150)
        y1 = random.randint(50, self.alto - 150)
        x2 = x1 + random.randint(50, 200)
        y2 = y1 + random.randint(50, 200)

        color = self.generar_color_aleatorio()
```

```
        if random.random() > 0.3:
            dibujo.ellipse([x1, y1, x2, y2],
                           fill=color,
                           outline=(0, 0, 0),
                           width=random.randint(1, 5))
        else:
            dibujo.ellipse([x1, y1, x2, y2], fill=color)

    def dibujar_linea(self, dibujo):
        """
        Dibuja una línea aleatoria
        """
        x1 = random.randint(10, self.ancho - 10)
        y1 = random.randint(10, self.alto - 10)
        x2 = random.randint(10, self.ancho - 10)
        y2 = random.randint(10, self.alto - 10)

        color = self.generar_color_aleatorio()
        ancho_linea = random.randint(1, 10)

        dibujo.line([x1, y1, x2, y2], fill=color, width=ancho_linea)

    def dibujar_poligono(self, dibujo):
        """
        Dibuja un polígono aleatorio (triángulo o cuadrilátero)
        """
        num_puntos = random.choice([3, 4])  # Triángulo o cuadrilátero
        puntos = []

        centro_x = random.randint(200, self.ancho - 200)
        centro_y = random.randint(200, self.alto - 200)
        radio = random.randint(50, 150)

        # Generar puntos alrededor de un centro
        for i in range(num_puntos):
            angulo = (2 * 3.14159 * i / num_puntos) + random.uniform(-0.2,
0.2)
            x = centro_x + radio * np.cos(angulo)
            y = centro_y + radio * np.sin(angulo)
            puntos.append((x, y))

        color = self.generar_color_aleatorio()
```

```python
        if random.random() > 0.3:
            dibujo.polygon(puntos, fill=color, outline=(0, 0, 0), width=2)
        else:
            dibujo.polygon(puntos, fill=color)

    def generar_imagen_aleatoria(self, num_figuras=None):
        """
        Genera una imagen completa con figuras aleatorias

        Args:
            num_figuras (int): Número de figuras a dibujar

        Returns:
            PIL.Image: Imagen generada
        """
        imagen = self.crear_imagen_vacia()
        dibujo = ImageDraw.Draw(imagen)

        if num_figuras is None:
            num_figuras = random.randint(5, 15)

        funciones_dibujo = {
            'rectangulo': self.dibujar_rectangulo,
            'elipse': self.dibujar_elipse,
            'linea': self.dibujar_linea,
            'poligono': self.dibujar_poligono
        }

        for _ in range(num_figuras):
            tipo_figura = random.choice(list(funciones_dibujo.keys()))
            funciones_dibujo[tipo_figura](dibujo)

        return imagen

    def generar_con_efectos(self, efecto='normal'):
        """
        Genera imagen con efectos especiales

        Args:
            efecto (str): Tipo de efecto ('normal', 'degradado', 'mosaico')
        """
        if efecto == 'normal':
            return self.generar_imagen_aleatoria()
```

```
        elif efecto == 'degradado':
            imagen = Image.new('RGB', (self.ancho, self.alto))
            dibujo = ImageDraw.Draw(imagen)

            for y in range(self.alto):
                r = int(255 * y / self.alto)
                g = int(255 * (1 - y / self.alto))
                b = int(255 * (0.5 + 0.5 * y / self.alto))
                for x in range(0, self.ancho, 10):
                    dibujo.rectangle([x, y, x+10, y+1], fill=(r, g, b))

            dibujo = ImageDraw.Draw(imagen)
            for _ in range(5):
                self.dibujar_elipse(dibujo)

            return imagen

        elif efecto == 'mosaico':
            imagen = Image.new('RGB', (self.ancho, self.alto))
            dibujo = ImageDraw.Draw(imagen)

            tamaño_celda = 40
            for x in range(0, self.ancho, tamaño_celda):
                for y in range(0, self.alto, tamaño_celda):
                    color = self.generar_color_aleatorio()
                    dibujo.rectangle([x, y, x+tamaño_celda-2,
y+tamaño_celda-2],
                                     fill=color)

            return imagen

# %%
# ==========================================
# FUNCIONES PARA VISUALIZAR EN NOTEBOOK
# ==========================================

def mostrar_imagen(imagen, titulo="Imagen Generada"):
    """
    Muestra una imagen en el notebook usando matplotlib

    Args:
        imagen (PIL.Image): Imagen a mostrar
```

```
        titulo (str): Título para la imagen
    """
    plt.figure(figsize=(10, 8))
    plt.imshow(imagen)
    plt.title(titulo, fontsize=16, fontweight='bold')
    plt.axis('off')  # Ocultar ejes
    plt.tight_layout()
    plt.show()

def mostrar_galeria(imagenes, filas=2, columnas=3):
    """
    Muestra múltiples imágenes en una galería

    Args:
        imagenes (list): Lista de imágenes PIL
        filas (int): Número de filas
        columnas (int): Número de columnas
    """
    fig, axes = plt.subplots(filas, columnas, figsize=(15, 10))
    axes = axes.flatten()  # Aplanar para fácil indexación

    for i, ax in enumerate(axes):
        if i < len(imagenes):
            ax.imshow(imagenes[i])
            ax.set_title(f"Imagen {i+1}", fontsize=12)
            ax.axis('off')
        else:
            ax.axis('off')  # Ocultar ejes vacíos

    plt.suptitle("Galería de Imágenes Generadas", fontsize=16,
fontweight='bold')
    plt.tight_layout()
    plt.show()

def guardar_imagen(imagen, nombre_base="imagen_generada"):
    """
    Guarda una imagen con timestamp

    Args:
        imagen (PIL.Image): Imagen a guardar
        nombre_base (str): Nombre base del archivo

    Returns:
```

```
        str: Ruta del archivo guardado
    """
    if not os.path.exists("imagenes_generadas"):
        os.makedirs("imagenes_generadas")

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    nombre_archivo = f"imagenes_generadas/{nombre_base}_{timestamp}.png"

    # Guardar imagen
    imagen.save(nombre_archivo, "PNG")
    print(f"✅ Imagen guardada: {nombre_archivo}")

    return nombre_archivo

# %%
# ============================================
# DEMOSTRACIÓN 1: Generación básica
# ============================================

print("DEMOSTRACIÓN DEL GENERADOR DE IMÁGENES")
print("=" * 60)

# Crear instancia del generador
generador = GeneradorImagenes(ancho=600, alto=400)

# Generar una imagen simple
print("\n1. Generando imagen aleatoria...")
imagen1 = generador.generar_imagen_aleatoria(num_figuras=8)
mostrar_imagen(imagen1, "Imagen Aleatoria Simple")

# Explicación de la imagen
print("\nCaracterísticas de la imagen:")
print(f"   • Dimensiones: {generador.ancho}x{generador.alto} píxeles")
print(f"   • Formato: RGB (24 bits por píxel)")
print(f"   • Figuras: 8 elementos aleatorios")
print(f"   • Tamaño aproximado: {generador.ancho * generador.alto * 3 /
1024 / 1024:.1f} MB")

# %%
# ============================================
# DEMOSTRACIÓN 2: Variaciones y estilos
# ============================================
```

```python
print("VARIACIONES DEL GENERADOR")
print("=" * 60)

# Generar imágenes con diferentes densidades
print("\nImágenes con diferente número de figuras:")

figuras_variadas = []
cantidades = [5, 10, 15, 20]

for i, cant in enumerate(cantidades):
    print(f"   Generando imagen con {cant} figuras...")
    img = generador.generar_imagen_aleatoria(num_figuras=cant)
    figuras_variadas.append(img)

# Mostrar en cuadrícula
fig, axes = plt.subplots(2, 2, figsize=(12, 12))
axes = axes.flatten()

for i, (ax, img, cant) in enumerate(zip(axes, figuras_variadas,
cantidades)):
    ax.imshow(img)
    ax.set_title(f"{cant} figuras", fontsize=14, fontweight='bold')
    ax.axis('off')

plt.suptitle("Variación por densidad de figuras", fontsize=16,
fontweight='bold')
plt.tight_layout()
plt.show()

# %%
# ============================================
# DEMOSTRACIÓN 3: Efectos especiales
# ============================================

print("EFECTOS ESPECIALES")
print("=" * 60)

# Generar diferentes efectos
efectos = ['normal', 'degradado', 'mosaico']
imagenes_efectos = []

for efecto in efectos:
    print(f"   Generando efecto: {efecto}")
```

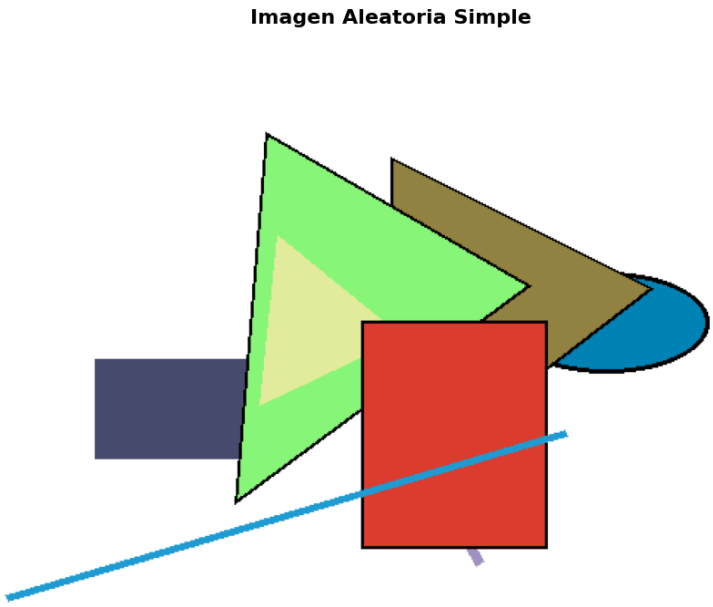
```
    img = generador.generar_con_efectos(efecto)
    imagenes_efectos.append(img)

# Mostrar efectos
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, (ax, img, efecto) in enumerate(zip(axes, imagenes_efectos,
efectos)):
    ax.imshow(img)
    ax.set_title(f"Efecto: {efecto.capitalize()}", fontsize=14,
fontweight='bold')
    ax.axis('off')

plt.suptitle("Diferentes efectos visuales", fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()

# %%
# ============================================
# DEMOSTRACIÓN 4: Galería de imágenes
# ============================================

print("GALERÍA DE IMÁGENES")
print("=" * 60)


num_imagenes = 6
galeria = []

print(f"Generando {num_imagenes} imágenes únicas...")

for i in range(num_imagenes):
    print(f"   Imagen {i+1}/{num_imagenes}")

    if i % 3 == 0:
        img = generador.generar_imagen_aleatoria(num_figuras=12)
    elif i % 3 == 1:
        img = generador.generar_con_efectos('degradado')
    else:
        img = generador.generar_con_efectos('mosaico')

    galeria.append(img)
```

```python
mostrar_galeria(galeria, filas=2, columnas=3)

print("\nGuardando una imagen de ejemplo...")
guardar_imagen(galeria[0], "galeria_ejemplo")

# %%
# ============================================
# BONUS: Mini-animación (secuencia de imágenes)
# ============================================

from IPython.display import Image as IPImage, display
import io

print("SECUENCIA ANIMADA")
print("=" * 60)

# Crear una secuencia de imágenes que cambian gradualmente
print("Generando secuencia de 8 imágenes...")

secuencia = []
for i in range(8):
    # Parámetros que cambian con i
    num_figuras = 5 + i * 2
    brillo = i * 30

    # Crear imagen con parámetros variables
    generador_temp = GeneradorImagenes(ancho=400, alto=300)
    imagen = generador_temp.crear_imagen_vacia()
    dibujo = ImageDraw.Draw(imagen)

    # Dibujar figuras con colores que cambian
    for _ in range(num_figuras):
        color = (brillo + random.randint(0, 50),
                 brillo + random.randint(0, 50),
                 brillo + random.randint(0, 50))
        x1 = random.randint(50, 350)
        y1 = random.randint(50, 250)
        x2 = x1 + 50
        y2 = y1 + 50
        dibujo.ellipse([x1, y1, x2, y2], fill=color)

    secuencia.append(imagen)
```

```python
# Mostrar secuencia
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.flatten()

for i, (ax, img) in enumerate(zip(axes, secuencia)):
    ax.imshow(img)
    ax.set_title(f"Frame {i+1}", fontsize=12)
    ax.axis('off')

plt.suptitle("Secuencia de imágenes (mini-animación)", fontsize=16,
fontweight='bold')
plt.tight_layout()
plt.show()

# %% [markdown]
# ## Conclusiones del Generador de Imágenes
#
# ### Logros alcanzados:
# - Creación exitosa de imágenes RGB desde cero
# - Implementación de múltiples tipos de figuras geométricas
# - Sistema de color aleatorio controlado
# - Visualización directa en Jupyter Notebook
# - Exportación a archivos PNG con timestamp
# - Múltiples efectos visuales (degradado, mosaico)
#
# ### Características técnicas implementadas:
# 1. **Generación procedural**: Cada imagen es única
# 2. **Control de parámetros**: Número de figuras, colores, tamaños
# 3. **Visualización integrada**: Uso de matplotlib para mostrar resultados
# 4. **Organización de archivos**: Carpeta automática con nombres únicos
# 5. **Interfaz interactiva**: Menú para explorar diferentes opciones
#
# ### Formatos y especificaciones:
# - **Formato de imagen**: PNG (Portable Network Graphics)
# - **Resolución**: Variable (400x300 por defecto)
# - **Profundidad de color**: 24 bits RGB
# - **Compresión**: Sin pérdida (lossless)
#
# ### Posibles mejoras:
# - Añadir texto a las imágenes
# - Implementar filtros artísticos (blur, sharpen)
# - Generar imágenes fractales
# - Exportar a otros formatos (JPG, GIF, BMP)
```

```
# - Crear GIFs animados con la secuencia
# - Añadir patrones y texturas
#
# ### Aplicaciones prácticas:
# - **Arte generativo**: Creación de obras únicas
# - **Testing**: Imágenes de prueba para algoritmos de visión
# - **Diseño**: Inspiración para patrones y composiciones
# - **Educación**: Enseñar conceptos de imagen digital
```

DEMOSTRACIÓN DEL GENERADOR DE IMÁGENES
==========
1. Generando imagen aleatoria...



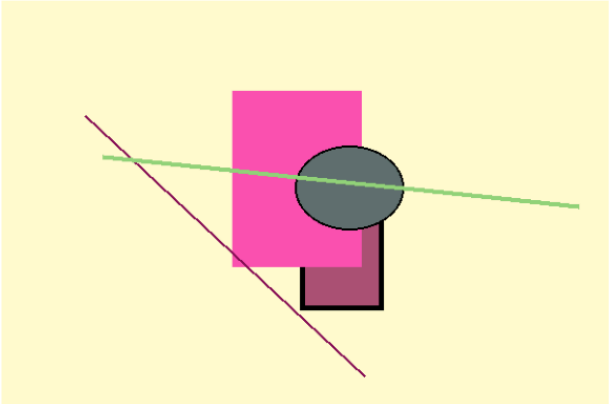
Características de la imagen:
- Dimensiones: 600x400 píxeles
- Formato: RGB (24 bits por píxel)
- Figuras: 8 elementos aleatorios
- Tamaño aproximado: 0.7 MB

VARIACIONES DEL GENERADOR
==========

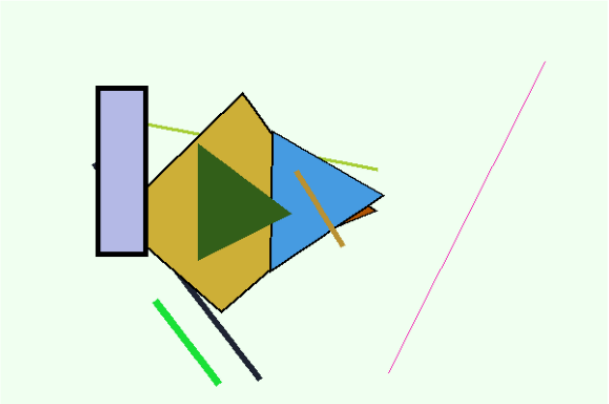
Imágenes con diferente número de figuras:
Generando imagen con 5 figuras...
Generando imagen con 10 figuras...
Generando imagen con 15 figuras...
Generando imagen con 20 figuras...

Variación por densidad de figuras

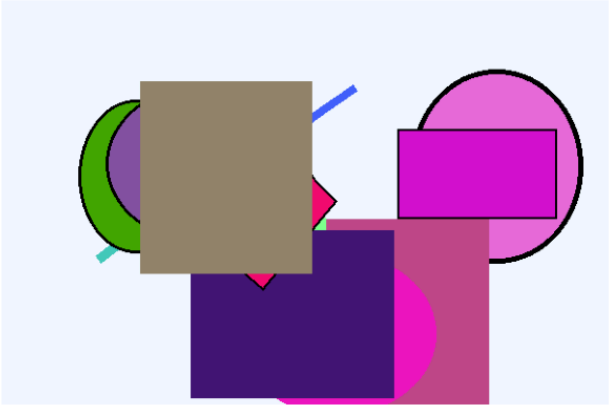
5 figuras



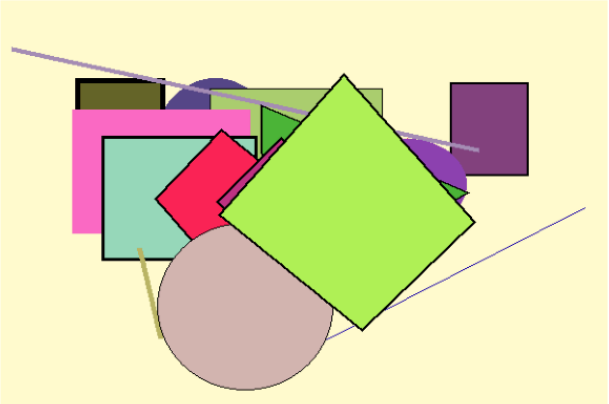
10 figuras



15 figuras



20 figuras



EFECTOS ESPECIALES

==========

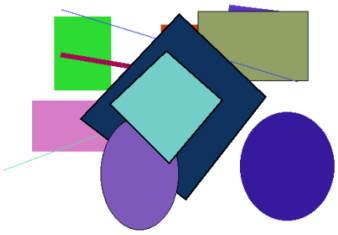
Generando efecto: normal

Generando efecto: degradado

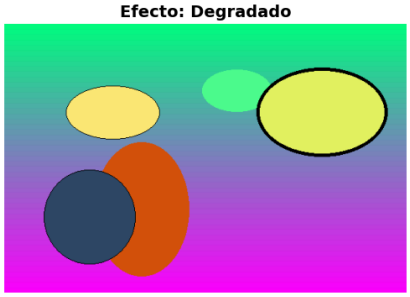
Generando efecto: mosaico

Diferentes efectos visuales

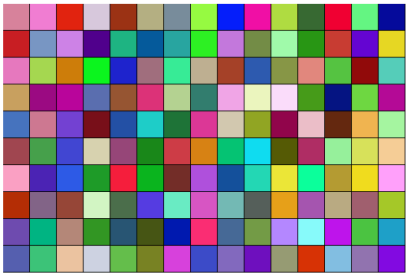
Efecto: Normal



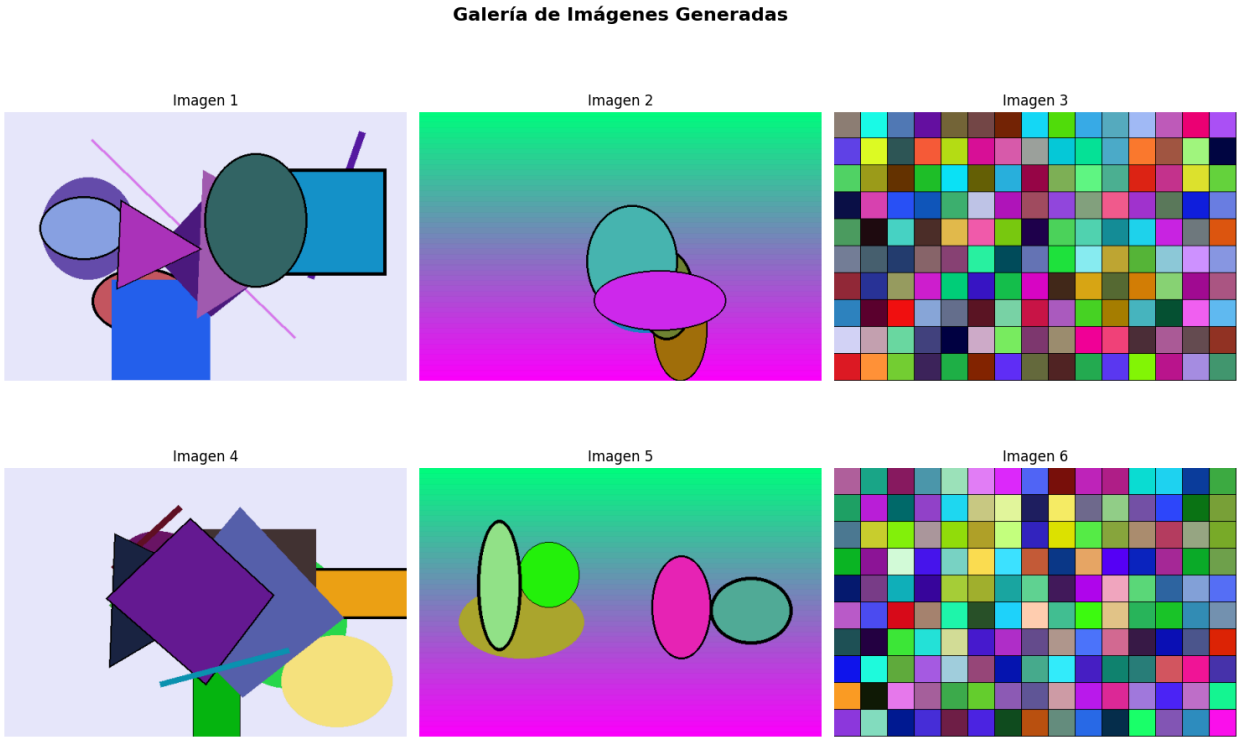
Efecto: Degradado



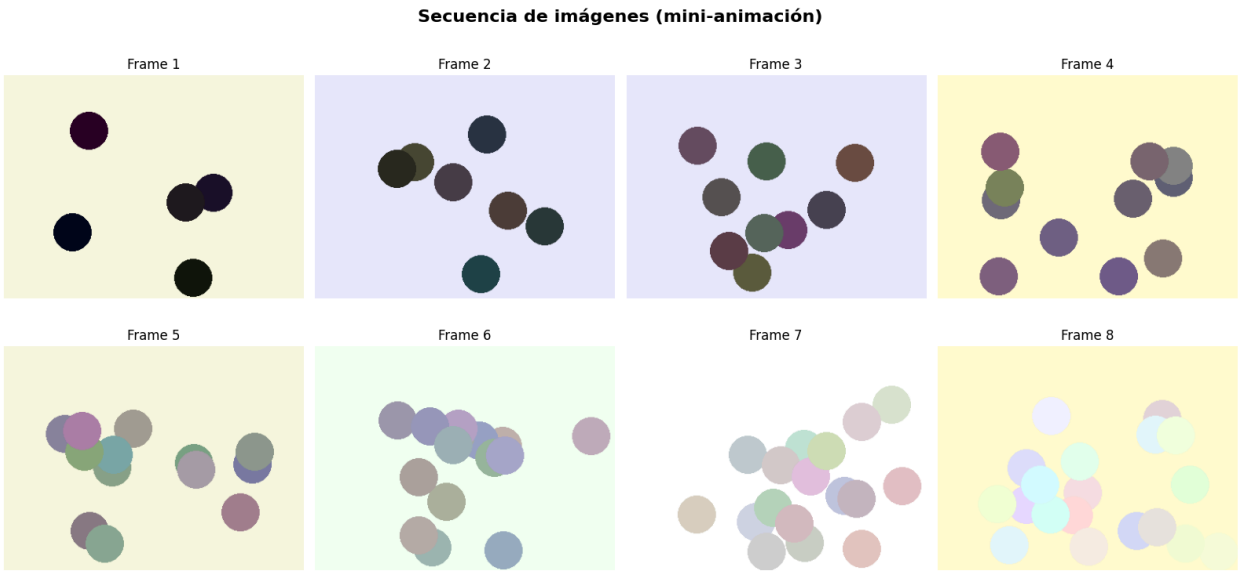
Efecto: Mosaico



GALERÍA DE IMÁGENES
==========
Generando 6 imágenes únicas...
  Imagen 1/6
  Imagen 2/6
  Imagen 3/6
  Imagen 4/6
  Imagen 5/6
  Imagen 6/6



Guardando una imagen de ejemplo…
Imagen guardada: imagenes_generadas/galeria_ejemplo_20260218_125902.png
SECUENCIA ANIMADA
==========
Generando secuencia de 8 imágenes…



GENERADOR DE DATOS SINTÉTICOS: Clientes y Ventas

```
# %% [markdown]
# # GENERADOR DE DATOS SINTÉTICOS: Clientes y Ventas
#
# ## Concepto
# Este notebook implementa un generador de datos sintéticos que crea
conjuntos de datos
# realistas para pruebas y análisis. Genera información de clientes,
productos y ventas
# con relaciones lógicas entre los datos.
#
# **Características:**
# - Generación de datos coherentes y realistas
# - Relaciones lógicas entre tablas (clientes → ventas → productos)
# - Distribuciones estadísticas realistas (edades, montos, fechas)
# - Múltiples formatos de exportación
# - Datos anonimizados pero consistentes
#
# **Formatos de salida:**
# - **CSV** (Comma-Separated Values): Formato universal para hojas de
cálculo
# - **JSON** (JavaScript Object Notation): Formato para APIs y aplicaciones
web
```

```
# - **Excel** (.xlsx): Formato para análisis en Excel
# - **SQL**: Scripts para bases de datos
#
# **Aplicaciones:**
# - Pruebas de software sin usar datos reales
# - Desarrollo y testing de bases de datos
# - Demostraciones y tutoriales
# - Prototipado rápido de aplicaciones
# - Entrenamiento de modelos de machine learning

# %%
# ============================================
# INSTALACIÓN Y VERIFICACIÓN DE LIBRERÍAS
# ============================================

import pandas as pd
import numpy as np
from faker import Faker
import random
from datetime import datetime, timedelta
import os
import matplotlib.pyplot as plt
import seaborn as sns


# Configuración de visualización
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
sns.set_style("whitegrid")

# %% [markdown]
# ## Librerías utilizadas
#
# #### 1. **pandas** (Data manipulation)
# - **Propósito**: Manipulación y análisis de datos estructurados
# - **Uso**: Crear DataFrames, exportar a CSV/Excel, filtrar datos
# - **Características**: Estructuras rápidas y flexibles (DataFrames)
# - **Instalación**: `pip install pandas`
#
# #### 2. **numpy** (Numerical computing)
# - **Propósito**: Operaciones numéricas eficientes
# - **Uso**: Generar distribuciones estadísticas, arrays multidimensionales
# - **Ventaja**: Cálculos vectorizados muy rápidos
```

```
# - **Instalación**: `pip install numpy`
#
# #### 3. **Faker** (Generación de datos falsos realistas)
# - **Propósito**: Crear datos ficticios realistas (nombres, direcciones,
emails)
# - **Uso**: Generar información de clientes creíble
# - **Instalación**: `pip install Faker`
#
# #### 4. **random** (estándar)
# - **Propósito**: Generación de números aleatorios
# - **Uso**: Selecciones aleatorias, probabilidades
#
# #### 5. **datetime** (estándar)
# - **Propósito**: Manejo de fechas y horas
# - **Uso**: Generar fechas de registro, compras, etc.
#
# #### 6. **matplotlib** / **seaborn** (Visualización)
# - **Propósito**: Visualizar estadísticas de los datos generados
# - **Uso**: Verificar que los datos tengan distribuciones realistas
(histogramas, diagramas de caja)
# - **Instalación**: `pip install matplotlib seaborn`
#
# #### 7. **openpyxl** (Excel)
# - **Propósito**: Exportar a formato Excel
# - **Uso**: Guardar datos en .xlsx para análisis en Excel
# - **Instalación**: `pip install openpyxl`

# %%
# ============================================
# CLASE PRINCIPAL: GeneradorDatosSinteticos
# ============================================

class GeneradorDatosSinteticos:
    """
    Generador completo de datos sintéticos para clientes, productos y
ventas
    """

    def __init__(self, idioma='es_ES', semilla=None):
        """
        Inicializa el generador con configuración regional

        Args:
```

```
            idioma (str): Código de idioma para Faker ('es_ES', 'en_US',
etc.)
            semilla (int): Semilla para reproducibilidad (None = aleatorio)
        """
        # Configurar Faker para datos realistas en español
        self.faker = Faker(idioma)

        # Semilla para reproducibilidad
        if semilla:
            Faker.seed(semilla)
            random.seed(semilla)
            np.random.seed(semilla)
            print(f"Semilla establecida: {semilla} (resultados
reproducibles)")

        # Categorías de productos
        self.categorias = {
            'Electrónica': ['Smartphone', 'Laptop', 'Tablet',
'Auriculares', 'Smartwatch'],
            'Hogar': ['Sofá', 'Mesa', 'Silla', 'Lámpara', 'Estantería'],
            'Ropa': ['Camiseta', 'Pantalón', 'Vestido', 'Chaqueta',
'Zapatos'],
            'Deportes': ['Bicicleta', 'Pesas', 'Esterilla', 'Balón',
'Raqueta'],
            'Libros': ['Novela', 'Ensayo', 'Manual', 'Cómic', 'Revista']
        }

        # Rangos de precios por categoría
        self.rangos_precios = {
            'Electrónica': (100, 1500),
            'Hogar': (50, 800),
            'Ropa': (20, 300),
            'Deportes': (30, 600),
            'Libros': (10, 100)
        }

        # Generar lista completa de productos
        self.productos = self._generar_catalogo_productos()

        print(f"Generador inicializado con {len(self.productos)} productos
en catálogo")

    def _generar_catalogo_productos(self):
```

```
        """
        Genera un catálogo de productos con IDs únicos
        """
        productos = []
        id_producto = 1

        for categoria, items in self.categorias.items():
            for nombre in items:
                precio_min, precio_max = self.rangos_precios[categoria]
                producto = {
                    'id_producto': id_producto,
                    'nombre': nombre,
                    'categoria': categoria,
                    'precio_unitario': round(random.uniform(precio_min,
precio_max), 2),
                    'stock': random.randint(10, 500),
                    'proveedor': self.faker.company()
                }
                productos.append(producto)
                id_producto += 1

        return productos

    def generar_clientes(self, n_clientes=100):
        """
        Genera una lista de clientes sintéticos

        Args:
            n_clientes (int): Número de clientes a generar

        Returns:
            pd.DataFrame: DataFrame con datos de clientes
        """
        print(f"\nGenerando {n_clientes} clientes...")
        clientes = []

        for i in range(1, n_clientes + 1):
            # Datos básicos
            nombre = self.faker.first_name()
            apellido = self.faker.last_name()

            # Generar email basado en el nombre
            email =
```

```
f"{nombre.lower()}.{apellido.lower()}@{self.faker.free_email_domain()}"

            # Teléfono (formato español)
            telefono = self.faker.phone_number()

            # Edad con distribución realista (más gente entre 25-55)
            edad = int(np.random.normal(40, 15))
            edad = max(18, min(90, edad))  # Limitar entre 18 y 90

            # Dirección completa
            direccion = self.faker.street_address()
            ciudad = self.faker.city()
            provincia = self.faker.province() if hasattr(self.faker,
'province') else self.faker.state()
            codigo_postal = self.faker.postcode()

            # Fecha de registro (últimos 3 años)
            dias_atras = random.randint(0, 1095)  # 3 años
            fecha_registro = datetime.now() - timedelta(days=dias_atras)

            # Segmento de cliente
            if edad < 25:
                segmento = random.choices(['Joven', 'Estudiante'],
weights=[0.7, 0.3])[0]
            elif edad < 40:
                segmento = random.choices(['Profesional', 'Familia'],
weights=[0.6, 0.4])[0]
            elif edad < 60:
                segmento = random.choices(['Familia', 'Senior'],
weights=[0.5, 0.5])[0]
            else:
                segmento = 'Senior'

            # Cliente VIP? (10% son VIP)
            es_vip = random.random() < 0.1

            cliente = {
                'id_cliente': i,
                'nombre_completo': f"{nombre} {apellido}",
                'email': email,
                'telefono': telefono,
                'edad': edad,
                'genero': random.choice(['M', 'F', 'Otro']),
```

```
                'direccion': direccion,
                'ciudad': ciudad,
                'provincia': provincia,
                'codigo_postal': codigo_postal,
                'fecha_registro': fecha_registro.strftime('%Y-%m-%d'),
                'segmento': segmento,
                'vip': es_vip,
                'activo': random.random() < 0.95  # 95% activos
            }

            clientes.append(cliente)

        # Crear DataFrame
        df_clientes = pd.DataFrame(clientes)
        print(f"{len(df_clientes)} clientes generados")

        return df_clientes

    def generar_ventas(self, df_clientes, n_ventas=500, fecha_inicio=None,
fecha_fin=None):
        """
        Genera un registro de ventas asociadas a clientes y productos

        Args:
            df_clientes (pd.DataFrame): DataFrame con clientes
            n_ventas (int): Número de ventas a generar
            fecha_inicio (datetime): Fecha más antigua (None = hace 1 año)
            fecha_fin (datetime): Fecha más reciente (None = hoy)

        Returns:
            pd.DataFrame: DataFrame con ventas
        """
        print(f"\nGenerando {n_ventas} ventas...")

        # Configurar fechas
        if fecha_fin is None:
            fecha_fin = datetime.now()
        if fecha_inicio is None:
            fecha_inicio = fecha_fin - timedelta(days=365)

        ventas = []

        # Clientes activos para ventas
```

```
        clientes_activos = df_clientes[df_clientes['activo'] ==
True]['id_cliente'].tolist()

        for i in range(1, n_ventas + 1):
            # Seleccionar cliente aleatorio (con peso por segmento)
            cliente_id = random.choice(clientes_activos)

            # Seleccionar producto aleatorio
            producto = random.choice(self.productos)

            # Cantidad (1-5 items, con distribución sesgada a 1-2)
            cantidad = int(np.random.exponential(1.5)) + 1
            cantidad = min(cantidad, 5)

            # Calcular importe
            precio_unitario = producto['precio_unitario']
            importe_total = round(precio_unitario * cantidad, 2)

            # Fecha aleatoria dentro del rango
            dias_rango = (fecha_fin - fecha_inicio).days
            dias_aleatorios = random.randint(0, dias_rango)
            fecha_venta = fecha_inicio + timedelta(days=dias_aleatorios)

            # Método de pago
            metodo_pago = random.choices(
                ['Tarjeta', 'Efectivo', 'Transferencia', 'PayPal'],
                weights=[0.6, 0.2, 0.15, 0.05]
            )[0]

            # Estado de la venta
            estado = random.choices(
                ['Completada', 'Pendiente', 'Cancelada', 'Devuelta'],
                weights=[0.85, 0.08, 0.05, 0.02]
            )[0]

            venta = {
                'id_venta': i,
                'id_cliente': cliente_id,
                'id_producto': producto['id_producto'],
                'producto_nombre': producto['nombre'],
                'categoria': producto['categoria'],
                'cantidad': cantidad,
                'precio_unitario': precio_unitario,
```

```
                'importe_total': importe_total,
                'fecha_venta': fecha_venta.strftime('%Y-%m-%d'),
                'metodo_pago': metodo_pago,
                'estado': estado,
                'descuento_aplicado': random.random() < 0.1  # 10% con
descuento
            }

            ventas.append(venta)

        # Ordenar por fecha
        df_ventas = pd.DataFrame(ventas)
        df_ventas =
df_ventas.sort_values('fecha_venta').reset_index(drop=True)

        print(f"{len(df_ventas)} ventas generadas")
        print(f"   Período: {fecha_inicio.strftime('%Y-%m-%d')} a
{fecha_fin.strftime('%Y-%m-%d')}")

        return df_ventas

    def get_catalogo_productos(self):
        """
        Devuelve el catálogo de productos como DataFrame
        """
        return pd.DataFrame(self.productos)

# %%
# ============================================
# FUNCIONES DE EXPORTACIÓN
# ============================================

def exportar_datos(df_clientes, df_ventas, df_productos, formato='todos',
nombre_base='datos_sinteticos'):
    """
    Exporta los datos generados a diferentes formatos

    Args:
        df_clientes (pd.DataFrame): Datos de clientes
        df_ventas (pd.DataFrame): Datos de ventas
        df_productos (pd.DataFrame): Catálogo de productos
        formato (str): 'csv', 'excel', 'json', 'sql' o 'todos'
        nombre_base (str): Nombre base para los archivos
```

```
    """
    # Crear carpeta de salida
    carpeta_salida =
f"datos_generados_{datetime.now().strftime('%Y%m%d_%H%M%S')}"
    os.makedirs(carpeta_salida, exist_ok=True)

    print(f"\nExportando datos a carpeta: {carpeta_salida}/")

    if formato in ['csv', 'todos']:
        # CSV
        df_clientes.to_csv(f"{carpeta_salida}/{nombre_base}_clientes.csv",
index=False, encoding='utf-8')
        df_ventas.to_csv(f"{carpeta_salida}/{nombre_base}_ventas.csv",
index=False, encoding='utf-8')

df_productos.to_csv(f"{carpeta_salida}/{nombre_base}_productos.csv",
index=False, encoding='utf-8')
        print("CSV exportados")

    if formato in ['excel', 'todos']:
        # Excel (múltiples hojas)
        with pd.ExcelWriter(f"{carpeta_salida}/{nombre_base}.xlsx",
engine='openpyxl') as writer:
            df_clientes.to_excel(writer, sheet_name='Clientes',
index=False)
            df_ventas.to_excel(writer, sheet_name='Ventas', index=False)
            df_productos.to_excel(writer, sheet_name='Productos',
index=False)
        print("Excel exportado")

    if formato in ['json', 'todos']:
        # JSON

df_clientes.to_json(f"{carpeta_salida}/{nombre_base}_clientes.json",
orient='records', indent=2, force_ascii=False)
        df_ventas.to_json(f"{carpeta_salida}/{nombre_base}_ventas.json",
orient='records', indent=2, force_ascii=False)

df_productos.to_json(f"{carpeta_salida}/{nombre_base}_productos.json",
orient='records', indent=2, force_ascii=False)
        print("JSON exportados")

    if formato in ['sql', 'todos']:
```

```
        # Script SQL
        with open(f"{carpeta_salida}/{nombre_base}_inserts.sql", 'w',
encoding='utf-8') as f:
            # Crear tablas
            f.write("-- SCRIPT DE INSERCIÓN DE DATOS SINTÉTICOS\n\n")

            # Productos
            f.write("-- TABLA productos\n")
            for _, row in df_productos.iterrows():
                f.write(f"INSERT INTO productos VALUES
({row['id_producto']}, '{row['nombre']}', '{row['categoria']}',
{row['precio_unitario']}, {row['stock']}, '{row['proveedor']}');\n")

            # Clientes
            f.write("\n-- TABLA clientes\n")
            for _, row in df_clientes.iterrows():
                f.write(f"INSERT INTO clientes VALUES ({row['id_cliente']},
'{row['nombre_completo']}', '{row['email']}', {row['edad']},
'{row['ciudad']}', '{row['fecha_registro']}', {row['vip']});\n")

            # Ventas
            f.write("\n-- TABLA ventas\n")
            for _, row in df_ventas.iterrows():
                f.write(f"INSERT INTO ventas VALUES ({row['id_venta']},
{row['id_cliente']}, {row['id_producto']}, {row['cantidad']},
{row['importe_total']}, '{row['fecha_venta']}', '{row['metodo_pago']}',
'{row['estado']}');\n")

        print("Script SQL generado")

    print(f"\nTodos los archivos guardados en: {carpeta_salida}/")
    return carpeta_salida

# %%
# ============================================
# FUNCIONES DE VISUALIZACIÓN Y ANÁLISIS
# ============================================

def analizar_datos(df_clientes, df_ventas, df_productos):
    """
    Realiza un análisis básico de los datos generados
    """
    print("=" * 70)
```

```
    print("ANÁLISIS DE DATOS GENERADOS")
    print("=" * 70)

    # Estadísticas de clientes
    print("\nESTADÍSTICAS DE CLIENTES")
    print("-" * 40)
    print(f"Total clientes: {len(df_clientes)}")
    print(f"Edad media: {df_clientes['edad'].mean():.1f} años")
    print(f"Clientes VIP: {df_clientes['vip'].sum()}
({df_clientes['vip'].mean()*100:.1f}%)")
    print(f"Clientes activos: {df_clientes['activo'].sum()}
({df_clientes['activo'].mean()*100:.1f}%)")
    print("\nSegmentos:")
    print(df_clientes['segmento'].value_counts())

    # Estadísticas de ventas
    print("\nESTADÍSTICAS DE VENTAS")
    print("-" * 40)
    print(f"Total ventas: {len(df_ventas)}")
    print(f"Importe total: {df_ventas['importe_total'].sum():,.2f} €")
    print(f"Ticket medio: {df_ventas['importe_total'].mean():.2f} €")
    print(f"Venta máxima: {df_ventas['importe_total'].max():.2f} €")
    print("\nMétodos de pago:")
    print(df_ventas['metodo_pago'].value_counts())
    print("\nEstados de venta:")
    print(df_ventas['estado'].value_counts())

    # Productos
    print("\nESTADÍSTICAS DE PRODUCTOS")
    print("-" * 40)
    print(f"Total productos: {len(df_productos)}")
    print(f"Precio medio: {df_productos['precio_unitario'].mean():.2f} €")
    print("\nCategorías:")
    print(df_productos['categoria'].value_counts())

def visualizar_datos(df_clientes, df_ventas):
    """
    Crea visualizaciones de los datos generados
    Versión corregida para pandas >= 2.0
    """
    fig = plt.figure(figsize=(16, 12))

    # 1. Distribución de edades
```

```
    ax1 = plt.subplot(2, 3, 1)
    df_clientes['edad'].hist(bins=20, edgecolor='black', alpha=0.7)
    ax1.set_title('Distribución de Edades')
    ax1.set_xlabel('Edad')
    ax1.set_ylabel('Frecuencia')

    # 2. Ventas por categoría
    ax2 = plt.subplot(2, 3, 2)
    ventas_cat =
df_ventas.groupby('categoria')['importe_total'].sum().sort_values()
    ventas_cat.plot(kind='barh', ax=ax2)
    ax2.set_title('Ventas por Categoría')
    ax2.set_xlabel('Importe Total (€)')

    # 3. Evolución temporal de ventas (CORREGIDO)
    ax3 = plt.subplot(2, 3, 3)
    df_ventas['fecha_venta'] = pd.to_datetime(df_ventas['fecha_venta'])

    # CORRECCIÓN: Cambiar 'M' por 'ME' (Month End)
    ventas_por_mes =
df_ventas.set_index('fecha_venta').resample('ME')['importe_total'].sum()

    ventas_por_mes.plot(ax=ax3, marker='o')
    ax3.set_title('Evolución Mensual de Ventas')
    ax3.set_xlabel('Fecha')
    ax3.set_ylabel('Ventas (€)')
    plt.xticks(rotation=45)

    # 4. Métodos de pago
    ax4 = plt.subplot(2, 3, 4)
    df_ventas['metodo_pago'].value_counts().plot(kind='pie',
autopct='%1.1f%%', ax=ax4)
    ax4.set_title('Métodos de Pago')

    # 5. Distribución de tickets
    ax5 = plt.subplot(2, 3, 5)
    df_ventas['importe_total'].hist(bins=30, edgecolor='black', alpha=0.7)
    ax5.set_title('Distribución de Importes')
    ax5.set_xlabel('Importe (€)')
    ax5.set_ylabel('Frecuencia')

    # 6. Clientes por ciudad (top 10)
    ax6 = plt.subplot(2, 3, 6)
```

```
    df_clientes['ciudad'].value_counts().head(10).plot(kind='bar', ax=ax6)
    ax6.set_title('Top 10 Ciudades')
    ax6.set_xlabel('Ciudad')
    ax6.set_ylabel('Nº Clientes')
    plt.xticks(rotation=45)

    plt.suptitle('Visualización de Datos Sintéticos Generados',
fontsize=16, fontweight='bold')
    plt.tight_layout()
    plt.show()

# %%
# ============================================
# GENERACIÓN PRINCIPAL DE DATOS
# ============================================

print("INICIANDO GENERACIÓN DE DATOS SINTÉTICOS")
print("=" * 70)

# Crear generador (con semilla fija para reproducibilidad)
generador = GeneradorDatosSinteticos(idioma='es_ES', semilla=42)

# 1. Generar catálogo de productos
df_productos = generador.get_catalogo_productos()
print(f"\nCatálogo de productos: {len(df_productos)} productos")
display(df_productos.head(10))

# 2. Generar clientes
df_clientes = generador.generar_clientes(n_clientes=200)
print(f"\nMuestra de clientes:")
display(df_clientes.head(10))

# 3. Generar ventas
df_ventas = generador.generar_ventas(df_clientes, n_ventas=1000)
print(f"\nMuestra de ventas:")
display(df_ventas.head(10))

# %%
# ============================================
# ANÁLISIS Y VISUALIZACIÓN
# ============================================

# Análisis estadístico
```

```python
analizar_datos(df_clientes, df_ventas, df_productos)

# Visualizaciones
visualizar_datos(df_clientes, df_ventas)

# Análisis adicional: Top clientes
print("\nTOP 10 CLIENTES POR VOLUMEN DE COMPRA")
print("-" * 50)
top_clientes = df_ventas.groupby('id_cliente').agg({
    'importe_total': 'sum',
    'id_venta': 'count'
}).rename(columns={'importe_total': 'gasto_total', 'id_venta':
'num_compras'})
top_clientes =
top_clientes.join(df_clientes.set_index('id_cliente')[['nombre_completo',
'segmento', 'vip']])
top_clientes = top_clientes.sort_values('gasto_total',
ascending=False).head(10)
display(top_clientes)

# Análisis de productos más vendidos
print("\nTOP 10 PRODUCTOS MÁS VENDIDOS")
print("-" * 50)
top_productos = df_ventas.groupby(['id_producto', 'producto_nombre',
'categoria']).agg({
    'cantidad': 'sum',
    'importe_total': 'sum'
}).sort_values('cantidad', ascending=False).head(10)
display(top_productos)

# %%
# ============================================
# EXPORTACIÓN A DIFERENTES FORMATOS
# ============================================

print("EXPORTACIÓN DE DATOS")
print("=" * 70)

# Preguntar formato (interactivo)
print("\nFormatos disponibles:")
print("1. CSV (universal)")
print("2. Excel (múltiples hojas)")
print("3. JSON (web/APIs)")
```

```python
print("4. SQL (base de datos)")
print("5. TODOS los formatos")

opcion = input("\nSelecciona formato (1-5): ")

formatos_map = {
    '1': 'csv',
    '2': 'excel',
    '3': 'json',
    '4': 'sql',
    '5': 'todos'
}

if opcion in formatos_map:
    carpeta = exportar_datos(
        df_clientes,
        df_ventas,
        df_productos,
        formato=formatos_map[opcion],
        nombre_base='tienda_sintetica'
    )

    # Mostrar contenido de la carpeta
    print(f"\nArchivos generados:")
    for archivo in os.listdir(carpeta):
        tamaño = os.path.getsize(os.path.join(carpeta, archivo)) / 1024  #
KB
        print(f"   • {archivo} ({tamaño:.1f} KB)")
else:
    print("Opción no válida")

# %%
# ============================================
# EJEMPLOS DE CONSULTAS ANALÍTICAS
# ============================================

print("EJEMPLOS DE CONSULTAS CON LOS DATOS GENERADOS")
print("=" * 70)

# Convertir fechas
df_ventas['fecha_venta'] = pd.to_datetime(df_ventas['fecha_venta'])

# 1. Ventas por mes y categoría
```

```python
print("\n1 VENTAS POR MES Y CATEGORÍA")
print("-" * 50)
df_ventas['mes'] = df_ventas['fecha_venta'].dt.to_period('M')
ventas_mensuales = df_ventas.pivot_table(
    values='importe_total',
    index='mes',
    columns='categoria',
    aggfunc='sum',
    fill_value=0
)
display(ventas_mensuales.round(2))

# 2. Tasa de repetición de clientes
print("\n2 ANÁLISIS DE CLIENTES")
print("-" * 50)
compras_por_cliente = df_ventas.groupby('id_cliente').size()
clientes_frecuentes = compras_por_cliente[compras_por_cliente > 1].count()
print(f"Clientes con más de 1 compra: {clientes_frecuentes}
({clientes_frecuentes/len(df_clientes)*100:.1f}%)")
print(f"Compras promedio por cliente: {compras_por_cliente.mean():.1f}")

# 3. Días entre compras (para clientes con múltiples compras)
print("\n3 FRECUENCIA DE COMPRA")
print("-" * 50)
df_ventas_ordenado = df_ventas.sort_values(['id_cliente', 'fecha_venta'])
df_ventas_ordenado['dias_entre_compras'] =
df_ventas_ordenado.groupby('id_cliente')['fecha_venta'].diff().dt.days
print(f"Días promedio entre compras:
{df_ventas_ordenado['dias_entre_compras'].mean():.1f} días")
print(f"Mediana días entre compras:
{df_ventas_ordenado['dias_entre_compras'].median():.1f} días")

# 4. Productos frecuentemente comprados juntos (simple)
print("\n4 PRODUCTOS POPULARES POR CATEGORÍA")
print("-" * 50)
for categoria in df_ventas['categoria'].unique():
    ventas_cat = df_ventas[df_ventas['categoria'] == categoria]
    top_producto =
ventas_cat.groupby('producto_nombre')['cantidad'].sum().idxmax()
    print(f"{categoria}: {top_producto}")
```

INICIANDO GENERACIÓN DE DATOS SINTÉTICOS
==========
Semilla establecida: 42 (resultados reproducibles)
Generador inicializado con 25 productos en catálogo

Catálogo de productos: 25 productos

	id_producto	nombre	categoria	precio_unitario	stock	proveedor
0	1	Smartphone	Electrónica	995.20	22	Banca Privada OLMJ S.L.N.E
1	2	Laptop	Electrónica	1138.17	135	Suministros BL S.Coop.
2	3	Tablet	Electrónica	412.50	387	Grupo Rivas S.A.T.
3	4	Auriculares	Electrónica	243.49	389	Alimentación Cuervo y asociados S.Com.
4	5	Smartwatch	Electrónica	1349.05	54	Fábrica Navarro S.A.
5	6	Sofá	Hogar	492.87	26	Caballero y Palomar S.L.L.
6	7	Mesa	Hogar	72.35	121	Banco Segovia & Asociados S.L.N.E
7	8	Silla	Hogar	224.50	318	Bermúdez y Marcos S.L.L.
8	9	Lámpara	Hogar	69.90	111	Fernandez & Asociados S.A.
9	10	Estantería	Hogar	587.01	369	Instalaciones Española S.L.L.

Generando 200 clientes...
200 clientes generados

	nombre_completo	email	genero	edad
0	Úrsula Parejo	úrsula.parejo@yahoo.com	M	47
1	Luna Font	luna.font@hotmail.com	M	37
2	Rosalina Murillo	rosalina.murillo@gmail.com	F	49
3	Rosario Balaguer	rosario.balaguer@hotmail.com	Otro	62

4	Clímaco Ayuso	clímaco.ayuso@hotmail.com	F	36
5	Lupe Torrecilla	lupe.torrecilla@gmail.com	M	36
6	Imelda Gutierrez	imelda.gutierrez@yahoo.com	F	63
7	Álvaro Cabo	álvaro.cabo@gmail.com	Otro	51
8	Gracia Calleja	gracia.calleja@gmail.com	F	32
9	Arsenio Acevedo	arsenio.acevedo@yahoo.com	F	48

Generando 1000 ventas...
1000 ventas generadas
  Período: 2025-02-18 a 2026-02-18

Muestra de ventas:

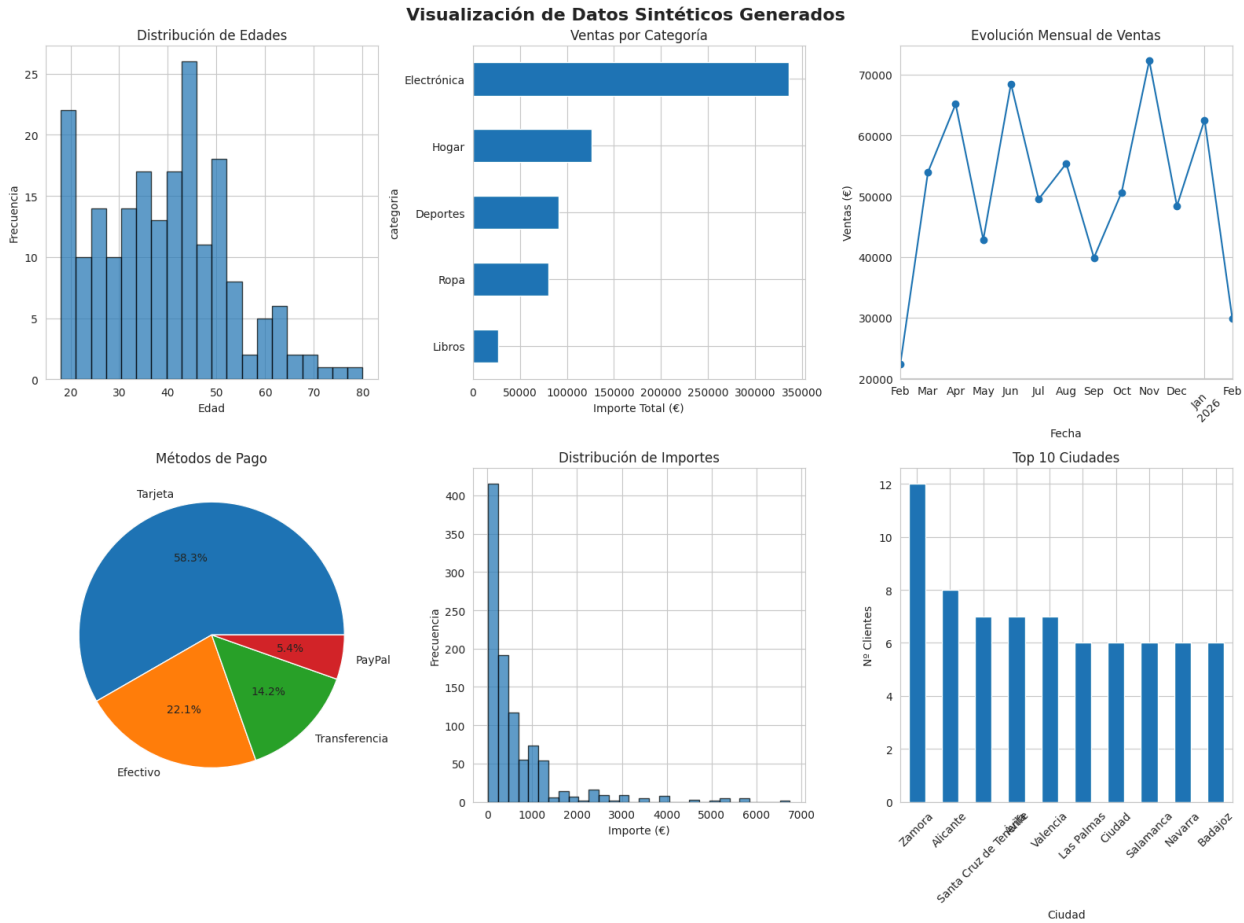
	cantidad	**precio_unitario**	**importe_total**
0	1	172.58	172.58
1	2	226.05	452.10
2	1	224.50	224.50
3	3	1138.17	3414.51
4	1	88.26	88.26
5	2	85.13	170.26
6	2	145.78	291.56
7	1	17.09	17.09
8	1	84.65	84.65
9	1	243.49	243.49

==========
ANÁLISIS DE DATOS GENERADOS
==========

ESTADÍSTICAS DE CLIENTES

Total clientes: 200
Edad media: 39.2 años
Clientes VIP: 17 (8.5%)
Clientes activos: 188 (94.0%)

```
Segmentos:
segmento
Senior        62
Familia       62
Profesional   44
Joven         23
Estudiante     9
Name: count, dtype: int64
ESTADÍSTICAS DE VENTAS
----------------------------------------
Total ventas: 1000
Importe total: 661,098.61 €
Ticket medio: 661.10 €
Venta máxima: 6745.25 €
Métodos de pago:
metodo_pago
Tarjeta          583
Efectivo         221
Transferencia    142
PayPal            54
Name: count, dtype: int64

Estados de venta:
estado
Completada    840
Pendiente      83
Cancelada      52
Devuelta       25
Name: count, dtype: int64

ESTADÍSTICAS DE PRODUCTOS
----------------------------------------
Total productos: 25
Precio medio: 323.79 €
Categorías:
categoria
Electrónica    5
Hogar          5
Ropa           5
Deportes       5
Libros         5
Name: count, dtype: int64
```



TOP 10 CLIENTES POR VOLUMEN DE COMPRA
---

gasto_total	num_compras	nombre_completo	segmento	vip	
id_cliente					
168	16904.97	5	Olalla Falcó	Senior	False
42	14279.03	10	Ángel Jara	Familia	False
194	11964.98	10	Calisto Goñi	Estudiante	False
147	11452.24	9	Encarna Hoyos	Estudiante	False
21	11169.03	6	Gloria Amores	Senior	False

172	10212.41	10	Mateo Castelló	Profesional	False
107	9527.61	5	María Cristina Agudo	Senior	False
94	9024.56	6	Guadalupe Echevarría	Profesional	False
73	8677.76	8	Jesusa Valencia	Familia	False
193	8343.57	7	Herminia Garcés	Senior	False

TOP 10 PRODUCTOS MÁS VENDIDOS
---

id_producto	producto_nombre	categoria	cantidad	importe_total
18	Esterilla	Deportes	110	9708.60
16	Bicicleta	Deportes	102	19215.78
10	Estantería	Hogar	99	58113.99
2	Laptop	Electrónica	96	109264.32
15	Zapatos	Ropa	91	19608.68
13	Vestido	Ropa	90	22197.60
19	Balón	Deportes	88	7491.44
20	Raqueta	Deportes	88	19892.40
12	Pantalón	Ropa	87	12682.86
25	Revista	Libros	87	7364.55

EXPORTACIÓN DE DATOS
==========

Formatos disponibles:
1. CSV (universal)
2. Excel (múltiples hojas)
3. JSON (web/APIs)
4. SQL (base de datos)
5. TODOS los formatos

Exportando datos a carpeta: datos_generados_20260218_172349/
CSV exportados

Excel exportado
JSON exportados
Script SQL generado

Todos los archivos guardados en: datos_generados_20260218_172349/

Archivos generados:
- tienda_sintetica_inserts.sql (124.0 KB)
- tienda_sintetica_clientes.json (79.7 KB)
- tienda_sintetica_productos.csv (1.4 KB)
- tienda_sintetica_clientes.csv (29.8 KB)
- tienda_sintetica_ventas.json (320.7 KB)
- tienda_sintetica_ventas.csv (76.7 KB)
- tienda_sintetica.xlsx (83.9 KB)
- tienda_sintetica_productos.json (4.2 KB)

```
/tmp/ipykernel_38675/3434799189.py:40: Pandas4Warning: The default 'epoch' date format is deprecated and will be removed in a future version, please use 'iso' date format instead.
  df_ventas.to_json(f"{carpeta_salida}/{nombre_base}_ventas.json", orient='records', indent=2, force_ascii=False)
```